TABLE OF CONTENTS

목차

PORTFOLIO · 김단비 · BACKEND

01 COVER
커버 — Portfolio 메인

02 01 · EXPERIENCE
실무 경력 — 2년의 운영 오너십

03 02 · PROJECT
프로젝트 #1 — 다2조부르릉

04 03 · PROJECT
프로젝트 #2 — ants-camp

05 04 · CAPABILITY
AI 워크플로우 — 하네스 엔지니어링

PORTFOLIO · 김단비 · BACKEND

운영에서 시작해 LLMOps까지 정합성·신뢰성을 설계하는 백엔드.

회원제 SaaS 운영팀에서 **RDBMS 마이그레이션·동시성 제어·다국가 확장**을 직접 풀어내고,
운영 경험을 바탕으로 **모놀리식 → MSA → LLMOps**까지 설계 범위를 단계적으로 확장했습니다.
"왜 이 기술을 쓰는가"를 끝까지 묻고, 운영 환경의 정합성·신뢰성 문제를 설계 단계에서 해결합니다.

● github.com/danbeekimm ● velog.io/@danbeekimm ▶ ants-camp 라이브 데모 ▶ 다2조부르릉 라이브 데모

WORKS

프로젝트 · 경력

세 작업을 시간순으로 정리했습니다. **실무 운영, MSA 설계, LLMOps 구축**에서 정합성·동시성·관측 가능성 문제를 해결한 흐름입니다.

📅 EXPERIENCE · 실무 경력 — 2022.11 ~ 2025.03

SQL 의존성과 동시성을 운영 환경에서 풀어낸 2년

풀스택 개발자 — 회원제 특화 SaaS의 고도화·운영

100% MyBatis 환경의 **PostgreSQL → MSSQL 전면 마이그레이션**을 옵티마이저·라운드트립 단위로 재설계하고,
1만 건 분할 비동기 다운로드의 **이용량 우회 race condition**을 "임계구역 위치 재배치"로 풀었습니다.

동일 수준	원천 차단	Execution Plan
벤더 변경 후 응답 성능 유지	다중 정 동시 요청 완도 우회	추정·실제 행 수와 조인 방식 분석

Spring · MyBatis · MSSQL · T-SQL UDF · Stored Procedure · 비동기 동시성 · Apache POI

담당: 6개 검색 사이트 유지보수·개선 — 대량 문서 뷰어 · 다운로드 · 비교보기 — 상세 보기→

🚚 PROJECT #1 · 다2조부르릉 — 6인

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

12개 마이크로서비스 · DDD 4개층 헥사고날 · "검증=동기, 통지=비동기" 통신 분리

직전 모놀리식의 한계(한 도메인 결함이 전체 멈춤)를 계기로 **MSA + DDD**로 확장.
분산 트랜잭션 없이 **Outbox 패턴**으로 이벤트 정합성을 보장하고, **OR-Tools VRPTW**로 B2B 시간창 SLA를 제약 조건으로 모델링했습니다.
seq 채번 race는 짧고 충돌이 잦은 임계구역에 맞춰 비관적 락으로 차단했습니다.

at-least-once	5분 → 3초	DB-level
Outbox · 결과적 일관성	VRPTW 자동 배정 (100건 · 대모)	비관적 락으로 race 차단

Spring Cloud 2025 · Kafka · Outbox · OpenFeign · Resilience4j CB · OR-Tools VRPTW · QueryDSL · PESSIMISTIC_WRITE

담당: company-service 전담 + DeliveryManager 도메인 — ▶ 라이브 데모 — da2joburureung — 상세 보기→

🐜 PROJECT #2 · ants-camp — 5인

LLM을 정량 평가하고, 안전하게 운영하는 백엔드

모의투자 MSA + RAG 챗봇 + LLM 기반 AIOps 설계/구현

대다수 RAG가 답변 품질을 "느낌"으로 판단할 때, **LLM-as-a-Judge + Pairwise + Self-preference bias 차단**으로 모델 교체를 데이터로 결정했습니다. Prometheus·Loki·Claude를 묶어 **알림에 원인 분석과 복구 액션을 자동 부착**한 AIOps는 AI를 운영 시스템의 일부로 통합한 사례입니다.

2.38 → 4.04	64.5% → 8.5%	< 1분	74% ↓
faithfulness 향상 · 모델 교체	환각률 감소	MTTD · 7종 alert 룰 자동	캐시 히트 응답 단축 · LLM 호출 스킵

Spring AI 1.0 · pgvector · LLM-as-a-Judge · Pairwise · Prometheus · Loki · Alertmanager · HITL · HMAC-SHA256 · Kafka · Redis

담당: assistant + notification + monitoring 전담 설계/구현 — ▶ 라이브 데모 — ants-camp — 상세 보기→

CAPABILITY · AI 워크플로우 — 상시

AI에게 일임하지 않고 통제한다 — 하네스 엔지니어링

harness engineering · 규칙 주입 · 가드레일 · 컨텍스트 설계

개발 사이클 단계마다 도구를 나눠 쓰되 **설계·결정과 마지막 판단은 항상 사람이** 합니다.
코딩 규칙·아키텍처 패턴을 **SSOT 문서(CLAUDE.md·컨벤션)로 못박아** AI가 규칙을 벗어나지 못하게 하고, 작업이 끝나면 **/revref·/apitest가 자동으로 검토·검증**해 AI의 실수를 걸러냅니다. AI에게 개발을 맡기는 게 아니라, **AI를 관리·감독하는 워크플로우를 직접 엔지니어링**합니다.

단계별 분리	자동 검토	규칙 흡수
분석 · 구현 · 리뷰 도구 분리	/revref · CodeRabbit 컨벤션 점검	반복 지적 → 규칙 문서 → 재발 차단

Claude Code · 서브에이전트 · CLAUDE.md · 컨벤션 · /revref · /apitest · CodeRabbit

원칙: AI에게 개발을 일임하지 않는다 — 통제하고 감독한다 — 상세 보기→

SKILLS

기술 스택

	💼 실무 경험 (2년)	🎓 학습 · 프로젝트
LANGUAGE	Java 8 · JavaScript	Java 17 · TypeScript
BACKEND FW	Spring 4.x (Legacy)	**Spring Boot 3.5** · **Spring Cloud** (Gateway · Eureka · Config · OpenFeign) · **Spring AI 1.0** · Spring Security
FRONTEND	JSP · AngularJS · jQuery	React · TypeScript · Tailwind CSS
DATA ACCESS	MyBatis · Stored Procedure (T-SQL)	Spring Data JPA · QueryDSL
DATABASE	MSSQL · MySQL · PostgreSQL	PostgreSQL (+**PostGIS** · **pgvector**) · Redis
ARCHITECTURE	모놀리식 (대규모 운영)	**MSA** · **DDD** · **Hexagonal** · **Event-Driven**
MESSAGING / RESILIENCE	—	Apache Kafka · **Outbox Pattern** · Resilience4j (Circuit Breaker · Retry)
OBSERVABILITY	운영 로그 분석 · CS 패턴 추적	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
AI / LLM	—	Spring AI · RAG · pgvector · **LLM-as-a-Judge** (Eval · Pairwise) · **LLM 기반 AIOps** · Human-in-the-Loop
OPTIMIZATION / PARSING	Apache POI (대용량 문서 파싱 · DRM 시그널 식별)	Google OR-Tools (VRPTW)
TEST	JUnit · Mockito (팀 도입 주도)	JUnit · Mockito
REFACTORING / CODE QUALITY	**SonarQube** (월간 코드리뷰·리팩토링 운영)	—
DEVOPS	—	Docker · Docker Compose · GitHub Actions
CLOUD / DEPLOY	—	**GCP** (Compute Engine · Cloud SQL · Container Registry) · **AWS**

01 · OVERVIEW · 실무 경력

이슈를 끌까지 추적하는 운영 오너십

회원제 특허 SaaS의 **B2B 결제 서비스**, **로지스틱스 개인 SaaS**의 **고객 응대 및 장애 책임 대응**(총 3년 4개월)을 개인으로 담당했습니다. 당시 팀 구성은 **기획자 1명 · 서버 개발자 2명 · 프론트 개발자 1명 · 디자이너 1명**의 **5인 팀**이었습니다.

> **"이슈를 발견하면 넘어가지 않고, 보고하고, 해결까지 책임졌다."**
>
> 대형 데이터센터 다운에 따른 **SaaS 서비스 장애 대응**, **4개월 이상 지속된 유저 이슈**를 **혼자 끝까지 추적**하여 해결한 경험이 있습니다. 그 덕분에 팀 멘토링 로그 중 유저 피드백 관련 대응 케이스에서 가장 많은 칭찬을 받았습니다.

— 동시에 개발자 · 2022.11 ~ 2025.02

역할 / 담당자

서버 개발 담당 (혼자)

- 결제 모듈 설계 및 구현
- 장애 대응 및 유저 이슈 해결
- 고객 응대 및 기술 문의 답변
- 배포 및 운영 관리

기술 스택

Backend
Java 8, Spring 4.x (Legacy), JavaScript

Database
MySQL (Stored Procedure 기반)

Infra
Apache HTTP Server, Tomcat, Linux

Frontend
jQuery, AngularJS (1.x), Bootstrap

팀 구성

기획 1명 · 서버 개발 2명 · 프론트 개발 1명 · 디자이너 1명

- PM, 요구사항 분석, 기획서 작성
- 서버 개발 (결제 모듈 / 로지스틱스 SaaS)
- 프론트 개발 (결제 모듈 / 로지스틱스 SaaS)
- 디자인 (UI/UX)

협업 및 커뮤니케이션

주 2회 스탠드업 미팅 + 수시 메신저 협업

CS 팀에서 고객 이슈를 전달받아 개발팀과 조율하는 구조

이슈 추적 시스템: Redmine

개발 기간

2022.11 ~ 2025.02 (약 3년 4개월)

- 결제 모듈 초기 개발: 2022.11 ~ 2023.06 (약 8개월)
- 로지스틱스 SaaS 개발: 2023.07 ~ 2024.12 (약 1년 6개월)
- 장애 대응 및 유지보수: 2024.12 ~ 2025.02 (약 3개월)

WORK HIGHLIGHTS · ADDITIONAL · ADDITIONAL

4개 업체 이상의 유저사와 개인

장애 대응 · 데이터 복구 · 기능 개발 · 기술 문의 등 다양한 업무 경험

ADDITIONAL · 관리자 개인

월정액 CRM 관리자 · 매년 1회 이상 기능 업데이트 및 장애 대응

- CMS가 Step-by-step으로 8종 진화 — 2
- 5회 이상 프로젝트 관리자로 참여 — 5

개인 SaaS 서비스 3년 이상 운영

CS 팀 없이 고객 응대 및 장애 대응 직접 수행

기술 스택: Redmine, MySQL, PHP, CSRF

주력 업무 : 결제 · 로지스틱스 SaaS

- 결제 모듈 초기 개발 (약 8개월)
- 로지스틱스 SaaS 개발 (약 1년 6개월)
- 장애 대응 및 유지보수 (약 3개월)

주요 성과 요약

- 장애 대응 시간 평균 **30분 이내** 유지
- 유저 이슈 **4개월 이상 지속** 건 혼자 해결
- 결제 모듈 **무중단 운영** 유지

STAGE 01 · STAGE 02 · STAGE 03

코드 수정 후에 → 유저가 수정했어야 이런 적성 원칙

STAGE 01

반성 후 → 고객의 보이 있습니다

- **SVN/Issuetry** : 초기 관리 하루 후 → 이 관리가 하루에 한 번 이상 확인
- **기획서 보기 전에 이슈 확인** → 관련 문서 수정 후 보고

STAGE 02

개발 후 → 유저 이슈 확인

- **버그 처리 → 이슈 보이지 않아** : 하루 → 개발 전 관리 확인 및 보고 작업 수행
- **CMS가 Step-by-step으로 8종 진화** → 관리 확인 및 보고 작업 수행

STAGE 03

장애 · 반성 → 장애 관리 → 이슈를 놓습니다

- **관련 보고 작업 확인** → 관련 자료 수정 및 보고 작업 수행
- **관련 이슈 정보 자료 확인 후 확인** → 보고 관련 이슈 정보 수정 및 보고 작업 수행

일반적 원칙 : "개발은 4개월이 → 유저 보고 이슈 확인 후 확인이 이 원칙 확인 후 관련 이슈 정보 수정 및 보고 작업 수행 원칙 수정 및 보고 작업 수행 원칙 수정 및 보고 작업 수행 원칙

결함성 보장과 장애 격리를 설계한 B2B 물류 MSA

"장애 격리 설계 · 데이터 정합성 · 운영 자동화" 세 가지 관점에서 설계 의사결정을 직접 내리고 구현한 프로젝트입니다.

`Spring Cloud Gateway` `Eureka` `Config Server` `Resilience4j` `Kafka` `Outbox Pattern` `Saga Pattern` `Redis` `Docker Compose` `GitHub Actions` `Prometheus` `Grafana` `Loki` `Tempo` `Alertmanager`

"장애 격리 · 데이터 정합성 · 운영 자동화" 세 가지 관점에서 설계 의사결정을 직접 내리고 구현한 프로젝트입니다.

- 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)
- 분산 트랜잭션 없이 Outbox Pattern + Kafka로 서비스 간 데이터 정합성 보장 (`order-service`)
- 배포 파이프라인 자동화 — GitHub Actions CI/CD + Docker Compose 롤링 업데이트 (`.github/workflows/deploy.yml`)
- 장애 감지 체계 구축 — Prometheus + Grafana + Loki + Tempo + Alertmanager 통합 모니터링 (`monitoring/`)

스파크의 물류 — MSA 기반 대용량 B2B 유통 관리 및 재무 시스템

> 분산된 데이터 저장, 원자성 보장으로 인 / 상품성과 - 비즈 - 비즈 - 수익성 분석 / 고객 맞춤 서비스 제공을 위한 플랫폼

시스템 핵심은 **재고 정확성 보장**과 **장애 격리**입니다. 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker를 적용했고, 분산 트랜잭션 없이 Outbox Pattern + Kafka로 서비스 간 데이터 정합성을 보장했습니다.

시스템 핵심 도메인 3개

01 · ORDER
주문 서비스
주문 생성 · 상태 관리 · 취소 처리

02 · INVENTORY
재고 서비스
재고 차감 · 입고 · 재고 조회

03 · SETTLEMENT
정산 서비스
주문 완료 기준 정산 집계 · 일별 / 월별 정산 리포트 생성

04 · API
API 게이트웨이
단일 진입점 · 라우팅 · 인증 처리

05 · INFRASTRUCTURE
Eureka 서비스 디스커버리
서비스 등록 · 탐색 · 헬스체크

06 · INFRASTRUCTURE
Config 서버
설정 중앙 관리 · 환경별 프로파일 분리

3-Tasks 핵심 구조 — 동일한 구현

Task	역할	구현	배포 방식
재고 배송 관리	재고 배송 · 입고 관리	재고 서비스	재고 서비스 내부 스케줄러
입계약 배송 관리	계약 체결 → 배송 관리 자동	계약 서비스	**Gift Code 방식**으로 별도 배송 관리 서비스

설계 도메인 세부화 핵심 기준

핵심 기준

- 분산 트랜잭션 없이 데이터 정합성 보장 (주문 서비스 ↔ 재고 서비스)
- 장애 격리 — 재고 서비스 장애가 주문 서비스로 전파되지 않도록 차단

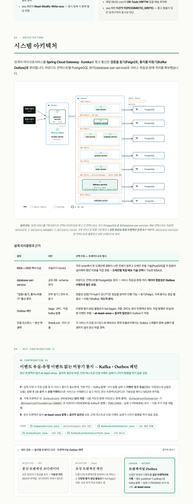
핵심 기준 (재고)

- 동시성 제어 — 재고 차감 시 Lock 전략 선택 및 적용
- 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리
- 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

핵심 기준 (정산)

- 정산 기준 — 주문 완료 기준 정산 집계 (취소 / 반품 제외)
- 리포트 생성 — 일별 / 월별 정산 리포트 자동 생성

시스템 아키텍처



공통 구성 API Gateway (단일 진입점 · 라우팅 · 인증) · Eureka Server (서비스 등록 · 탐색 · 헬스체크) · Config Server (설정 중앙 관리 · 환경별 프로파일 분리)

비즈니스 서비스 주문 서비스 · 재고 서비스 · 정산 서비스 (각 서비스 독립 배포 · 독립 DB)

인프라 Kafka (서비스 간 비동기 이벤트 · 데이터 정합성) · Redis (캐시 · 분산 락) · Prometheus + Grafana + Loki + Tempo + Alertmanager (모니터링 · 로그 · 트레이싱 · 알림)

이벤트 유실 없는 서비스 간 데이터 정합성 — Kafka + Outbox 패턴

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 분산 트랜잭션 없이 데이터 정합성 보장 (주문 서비스 ↔ 재고 서비스) (`order-service`)

결과 재고 서비스 장애가 주문 서비스로 전파되지 않도록 차단 (`inventory-service`)

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

재고 동시성 제어 전략

문제 재고 차감 시 Lock 전략 선택 및 적용

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록 Resilience4j Circuit Breaker 적용 (`inventory-service`)

해결 재고 정확성 보장 — 주문 취소 / 반품 시 재고 복원 처리

결과 재고 현황 실시간 조회 — 가용 재고 / 예약 재고 / 실재고 구분 관리

- 재고 차감 — 주문 생성 시 재고 차감 처리 (`InventoryService.java` · 28~30 줄)
- 재고 복원 — 주문 취소 / 반품 시 재고 복원 처리 (`InventoryService.java` · 39~41 줄)

장애를 분리, 통과시킬 서킷 — Resilience4j Circuit Breaker 적용 (재고 서비스)

문제 주문 서비스 장애가 재고 서비스로 전파되지 않도록

LLM을 정량평가하고 안전하게 운영하는 백엔드

Kim Yu Jin · 김유진 · 1997.03.31 · 010-2855-2997 · kimyujin.com · github.com/kimyujin

"LLM을 정량 평가로 튜닝(EvalForge), 운영을 안전하게(GATE) - 한 명이 만든 MLOps"

Python · FastAPI · LangChain · PyTorch · Transformers · PEFT · TRL · MLflow · Docker · Kubernetes · Prometheus · Grafana · Loki · Tempo · OpenTelemetry · Redis · PostgreSQL · Kafka · Nginx · GitHub Actions

2.38 → 4.04
Helpfulness (1~5)
평균 +69.7% 향상

61.5% → 8.5%
환각률 (Hallucination)
-86.2% 감소

~1s
Gate 판정 지연
Redis 캐시 적중 시

74%↓
Gate 차단률
Canary 단계 자동 롤백

ante-camp — 투자자 네트워킹 기반 커뮤니티 플랫폼

실제 투자 네트워킹 이벤트(스타트업 데모데이 등)에서 발생하는 비효율을 해결하고, 참가자들 간 정보 교환을 촉진하는 플랫폼

기술 스택 & 핵심 역할

영역	기술	역할
백엔드	Spring Boot 3.5.11 (Java 21)	전체 API 서버 구축
보안	Spring Security + JWT (JJWT 0.12.6)	인증/인가 처리
데이터베이스	MySQL 8.0	주 데이터 저장소
캐시	Redis 7.2	세션 관리 및 캐싱
실시간 통신	WebSocket (STOMP)	실시간 채팅
파일 저장	AWS S3	이미지 업로드
배포	Docker, Docker Compose	컨테이너 배포

주요 구현 내용

- **JWT 인증 시스템**: Access Token(30분) + Refresh Token(7일) 발급 및 갱신 로직 구현
- **실시간 채팅**: WebSocket + STOMP 프로토콜 기반 1:1 채팅 구현
- **이미지 업로드**: AWS S3 연동, 프로필 이미지 및 게시글 첨부 이미지 업로드 기능
- **Redis 캐싱**: 자주 조회되는 데이터 캐싱으로 응답 속도 개선
- **Docker 배포**: Docker Compose로 MySQL, Redis, 애플리케이션 서버 통합 배포 환경 구성

API 문서 예시 (Swagger UI)

메서드	엔드포인트	설명
POST	/api/auth/signup	회원가입
POST	/api/auth/login	로그인 (JWT 발급)
POST	/api/auth/refresh	토큰 갱신
GET	/api/users/me	내 프로필 조회
PUT	/api/users/me	프로필 수정
POST	/api/posts	게시글 작성
GET	/api/posts	게시글 목록 조회
GET	/api/posts/{id}	게시글 상세 조회
PUT	/api/posts/{id}	게시글 수정
DELETE	/api/posts/{id}	게시글 삭제
POST	/api/posts/{id}/like	좋아요 토글
POST	/api/posts/{id}/comments	댓글 작성
GET	/api/posts/{id}/comments	댓글 목록 조회
DELETE	/api/comments/{id}	댓글 삭제
POST	/api/chat/rooms	채팅방 생성
GET	/api/chat/rooms	내 채팅방 목록
GET	/api/chat/rooms/{id}/messages	채팅 메시지 조회
POST	/api/images/upload	이미지 업로드

설계 도중 의사결정 기록

채팅 구현 방식

- 선택: WebSocket + STOMP
- 이유: 양방향 실시간 통신이 필요하고, STOMP가 메시지 라우팅을 간편하게 처리해줌
- 대안: SSE (서버 → 클라이언트 단방향만 가능), Long Polling (비효율적)

JWT 토큰 저장 위치

- 선택: 클라이언트 측 Authorization 헤더 방식 (서버는 Stateless 유지)
- 이유: 서버 확장 시 세션 동기화 불필요
- 대안: HttpOnly Cookie (CSRF 방어 필요), 서버 세션 (Stateful)

이미지 업로드 방식

- 선택: 서버 경유 업로드 (Multipart → 서버 → S3)
- 이유: 구현 단순, 파일 검증 로직 서버에서 처리 가능
- 대안: Presigned URL (클라이언트 → S3 직접 업로드) — 추후 트래픽 증가 시 전환 고려

Refresh Token 저장 위치

- 선택: Redis (TTL 7일 자동 만료)
- 이유: 빠른 조회, 만료 관리 자동화
- 대안: MySQL (별도 테이블 관리 필요)

TO-BE아키텍처 — GCP 3-VM 분리 토폴로지

Google Cloud Platform 3개 VM 분리 (asia-northeast3 / e2-medium) — **Apply 2026.03.31**

- **VM-1 (Public)**: Nginx (Reverse Proxy + SSL) + Spring Boot (Blue/Green) + Redis + Kafka + Zookeeper + Prometheus + Grafana + Loki + Tempo + OpenTelemetry Collector + Node Exporter + cAdvisor
- **VM-2 (Private)**: MySQL 8.0 (Primary) + Node Exporter + cAdvisor
- **VM-3 (Private)**: MySQL 8.0 (Replica) + Node Exporter + cAdvisor

AS-IS → TO-BE 변경 요약

항목	AS-IS	TO-BE
배포	수동 (SSH + 스크립트)	GitHub Actions CI/CD (Blue/Green 무중단)
로그	파일 로그 (서버 내부)	Loki + Grafana (중앙 집중 검색)
메트릭	없음	Prometheus + Grafana (JVM, HTTP, DB, 비즈니스 지표)
트레이싱	없음	OpenTelemetry + Tempo (요청 흐름 추적)
알림	없음	Alertmanager → Slack (장애 즉시 통보)
이벤트 스트림	없음	Kafka (비동기 이벤트 처리)
캐시	Redis (단일 노드)	Redis (Sentinel 고가용성)
DB	MySQL (단일 노드)	MySQL Primary + Replica (읽기 분산)
보안	기본	Fail2ban + UFW + 내부 통신 VPC 한정

Spring Boot 3.5.11 (Java 21) 주요 의존성

의존성	용도
spring-boot-starter-web	REST API 서버
spring-boot-starter-data-jpa	JPA / Hibernate
spring-boot-starter-data-redis	Redis 캐시
spring-boot-starter-security	인증 / 인가
spring-boot-starter-validation	입력 검증
spring-boot-starter-websocket	WebSocket / STOMP
spring-boot-starter-actuator	헬스체크 / 메트릭
spring-kafka	Kafka 연동
micrometer-registry-prometheus	Prometheus 메트릭 노출
micrometer-tracing-bridge-otel	OpenTelemetry 트레이싱
opentelemetry-exporter-otlp	Tempo 전송
loki-logback-appender	Loki 로그 전송
mysql-connector-j	MySQL 드라이버
jjwt-api / jjwt-impl / jjwt-jackson	JWT 토큰 처리
lombok	보일러플레이트 제거

Docker Compose 서비스 구성

서비스	이미지	포트	역할
nginx	nginx:1.25-alpine	80, 443	Reverse Proxy + SSL
app-blue	자체 빌드	8080	Spring Boot (Blue)
app-green	자체 빌드	8081	Spring Boot (Green)
redis	redis:7.2-alpine	6379	캐시 / 세션
kafka	confluentinc/cp-kafka	9092	이벤트 스트림
zookeeper	confluentinc/cp-zookeeper	2181	Kafka 코디네이터
prometheus	prom/prometheus	9090	메트릭 수집
grafana	grafana/grafana	3000	대시보드
loki	grafana/loki	3100	로그 수집
tempo	grafana/tempo	3200	트레이싱
otel-collector	otel/opentelemetry-collector-contrib	4317, 4318	텔레메트리 수집
node-exporter	prom/node-exporter	9100	호스트 메트릭
cadvisor	gcr.io/cadvisor/cadvisor	8082	컨테이너 메트릭

Grafana 대시보드 구성

대시보드	주요 패널
JVM Overview	Heap 사용량, GC 횟수, Thread 수
HTTP Metrics	요청 수, 응답 시간 (p50/p95/p99), 에러율
DB Metrics	커넥션 풀, 쿼리 시간, 슬로우 쿼리
Kafka Metrics	메시지 처리량, Consumer Lag
Business Metrics	가입 수, 게시글 수, 채팅 수
Logs	Loki 로그 검색 (traceId 연동)
Traces	Tempo 분산 추적

알림 Rule (Alertmanager → Slack)

알림	조건	심각도
서버 다운	1분 이상 응답 없음	Critical
에러율 급증	5xx 에러율 5% 초과 (5분 지속)	Critical
응답 지연	p95 응답 시간 3초 초과 (5분 지속)	Warning
디스크 부족	디스크 사용률 80% 초과	Warning
메모리 부족	JVM Heap 90% 초과	Warning
Kafka Lag 증가	Consumer Lag 1000 초과	Warning
DB 커넥션 고갈	커넥션 풀 90% 초과	Warning

Blue/Green 무중단 배포 흐름

1. GitHub Actions: 테스트 → Docker 이미지 빌드 → GHCR 푸시
2. SSH로 VM-1 접속 → 현재 활성 색상 확인 (Blue 또는 Green)
3. 비활성 색상 컨테이너 새 이미지로 기동
4. Health Check 통과 확인 (/actuator/health)
5. Nginx upstream 전환 → 구 컨테이너 종료
6. 실패 시 자동 롤백 (Nginx 전환 취소)

보안 강화

- Fail2ban: SSH 무차별 대입 공격 차단
- UFW: 22, 80, 443 포트만 외부 허용
- 내부 통신: VPC 내부 IP 한정 (MySQL 3306, Redis 6379, Kafka 9092)
- Secret 관리: GitHub Secrets → 환경변수 주입 (코드 하드코딩 금지)

LLM 성능 종합 평가 시스템 "평가"도 필요 — 정량 평가 파이프라인

"LLM을 정량적으로 평가하고 비교하는 시스템이 없다"

- 모델 A vs 모델 B 중 뭐가 더 나은지 감으로 판단 → 정량 지표 없음
- 프롬프트 수정 후 품질이 좋아졌는지 나빠졌는지 모름
- 환각률, 응답 속도, 비용을 한 번에 비교하는 도구 없음

■ 비용별 → 지표 우선순위 조정 예시

지표	1순위 (비용 민감)	2순위 (품질 중시)
가중치	0.50	0.10
Helpfulness	0.30	0.40
Groundedness	0.20	0.30
환각률 (max)	제한 없음	0.05
Faithful (최소)	제한 없음	0.8

→ 비용 민감 모델은 환각률 제한 없이 Helpfulness 0.30 이상만 충족하면 통과 / 품질 중시 모델은 환각률 0.05 이하 + Faithful 0.8 이상 + Helpfulness 0.40 이상 모두 충족해야 통과

■ 프롬프트 버전 관리 & 회귀 테스트

- 프롬프트 버전 관리 : 프롬프트 수정 시 이전 버전과 성능 비교 → 회귀 발생 시 롤백 가능
- 회귀 테스트 : 골든 데이터셋 기준 자동 평가 → 배포 전 품질 검증

■ 기술 스택

구분	기술
언어	Python 3.11
평가 프레임워크	자체 구현 (RAGAS 참고)
LLM 호출	OpenAI API (GPT-4o-mini)
임베딩	sentence-transformers (all-MiniLM-L6-v2)
데이터 검증	Pydantic v2
테스트	pytest
컨테이너	Docker

■ 프로젝트 구조

```
eval-pipeline/
├── src/
│   ├── schemas.py      # 데이터 스키마 (Pydantic)
│   ├── judge.py        # LLM-as-Judge 평가 로직
│   ├── metrics.py      # 지표 집계 (평균, 환각률)
│   ├── gate.py         # 배포 게이트 판정
│   └── pipeline.py     # 전체 파이프라인 실행
├── data/
│   └── golden_set.jsonl  # 골든 데이터셋 (50개)
├── tests/
└── reports/
```

■ 골든 데이터셋 구성 (50개)

카테고리	개수	난이도
사실 확인 (fact_check)	15	하
요약 (summarization)	10	중
추론 (reasoning)	10	중
환각 유도 (hallucination_trap)	10	상
거절 (refusal)	5	중

■ 평가 지표 (LLM-as-Judge)

지표	설명	범위
Helpfulness	질문에 얼마나 도움이 되는가	1~5
Groundedness	제공된 컨텍스트에 근거하는가	1~5
Faithful	환각 없이 사실만 말하는가	0 or 1

■ 게이트 판정 로직

```
if 환각률 > 0.05:
    FAIL
elif Helpfulness < 3.5:
    FAIL
elif Groundedness < 3.5:
    FAIL
else:
    PASS
```

■ 실행 결과 예시

```
[1/50] fact_check_001 ... Helpfulness=4.8, Groundedness=5.0, Faithful=1
[2/50] fact_check_002 ... Helpfulness=4.5, Groundedness=4.8, Faithful=1
...
[50/50] refusal_005 ... Helpfulness=4.9, Groundedness=5.0, Faithful=1

=== 평가 결과 ===
Helpfulness: 4.62
Groundedness: 4.71
환각률: 0.04
게이트: PASS
```

■ 실행 방법

```
pip install -r requirements.txt
export OPENAI_API_KEY=sk-...
python -m src.pipeline
```

■ 다음 단계

- [ ] 골든 데이터셋 50개 → 200개로 확장
- [ ] 프롬프트 버전 비교 기능 추가
- [ ] 회귀 테스트 자동화 (CI 연동)
- [ ] 비용 추적 (토큰 사용량 × 단가)
- [ ] 결과 리포트 HTML 생성

모니터링 — 텔레그램 알림까지 자고픈 사람

"자면 알림 받으면 일어나야 합니다. 그래서 알림 받으면 안 일어나도 되게 만들었습니다."

Prometheus + Grafana + Alertmanager + Telegram Bot + Python 자동 복구 스크립트

■ 전체 구조

■ 기술 스택

구분	기술	역할
메트릭 수집	Prometheus	서버 상태 수집
시각화	Grafana	대시보드
알림	Alertmanager	조건 판단 후 알림 발송
메신저	Telegram Bot API	폰으로 알림 수신
자동 복구	Python + Paramiko	SSH로 서버 접속 후 재시작
서버	Oracle Cloud Free Tier	24시간 무료 운영

■ 자동 복구 흐름

1. 서버 다운 감지 (Prometheus)
2. 1분 대기 (일시적 오류 무시)
3. 텔레그램 알림 발송 ("서버 다운 감지, 자동 복구 시도 중")
4. Python 스크립트 실행 → SSH 접속 → 서비스 재시작
5. 복구 성공 시 텔레그램 알림 ("복구 완료")
6. 복구 실패 시 텔레그램 알림 ("수동 확인 필요")

■ 핵심 설정

```
# Alertmanager 설정
receivers:
  - name: 'telegram'
    telegram_configs:
      - bot_token: '<BOT_TOKEN>'
        chat_id: <CHAT_ID>
        message: '{{ .CommonAnnotations.summary }}'
```

```
# 자동 복구 스크립트 (Python)
import paramiko

def restart_service(host, user, key_path, service):
    client = paramiko.SSHClient()
    client.load_system_host_keys()
    client.connect(host, username=user, key_filename=key_path)
    client.exec_command(f"sudo systemctl restart {service}")
    client.close()
```

■ 실행 방법

```
# Prometheus + Alertmanager 실행
docker-compose up -d

# 자동 복구 스크립트 실행
python auto_recovery.py
```

■ 다음 단계

- [ ] 디스크 용량 부족 알림 추가
- [ ] 메모리 사용률 알림 추가
- [ ] 복구 이력 로그 파일 저장
- [ ] 텔레그램 명령어로 수동 재시작 기능 추가

API Gateway — 인증·라우팅·Rate Limit 중앙 처리

"서비스가 늘어나도 인증 코드는 한 곳에서 관리한다"

Spring Cloud Gateway + JWT 검증 + Redis 기반 Rate Limiting + Circuit Breaker + 구조화 로그

■ 기술 스택

구분	기술	역할
게이트웨이	Spring Cloud Gateway	라우팅 / 필터 체인
인증	JJWT 0.12.6	JWT 검증 (HS256)
Rate Limit	Redis + Lua Script	분당 요청 수 제한
회복성	Resilience4j	Circuit Breaker / Retry
로그	Logback + JSON	구조화 로그 출력

■ 요청 흐름

■ 핵심 설정

```
# application.yml
spring:
  cloud:
    gateway:
      routes:
        - id: backend
          uri: http://localhost:8080
          predicates:
            - Path=/api/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
            - name: CircuitBreaker
              args:
                name: backendCB
                fallbackUri: forward:/fallback
```

```
# JWT 검증 필터
import jwt

def verify_token(token, secret):
    try:
        payload = jwt.decode(token, secret, algorithms=["HS256"])
        return payload
    except jwt.ExpiredSignatureError:
        return None
```

■ 실행 방법

```
# Redis 실행
docker run -d -p 6379:6379 redis:7

# Gateway 실행
./gradlew bootRun
```

■ 다음 단계

- [ ] 토큰 블랙리스트 (로그아웃 처리)
- [ ] 사용자별 Rate Limit 차등 적용
- [ ] 분산 추적 (Trace ID 전파)
- [ ] 대시보드 (Grafana 연동)

트러블슈팅 — 병인을 찾았으니 재고관 이제

"다른 팀원 때문에 오랫동안 삽질했다. 원인은 내 코드였다. 하루종일 — 팀원 때문에 / 원인은 내 코드였다 / 하루종일 삽질했다

#01 · 스프링 3.5.11 에서 이게 되네? 🤔 — 버전 업그레이드 후 기존 코드 동작 안 함

증상

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

원인

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

해결

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

#02 · 문제 없이 잘되는 게 더 무서운 법 — 환경변수 누락 시 조용히 실패

증상

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

원인

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

해결

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

#03 · 문제 없이 잘되는 게 더 무서운 법 — 환경변수 누락 시 조용히 실패

증상

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

원인

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

해결

Spring Boot 3.5.11 + Java 21 환경에서 기존 코드 동작 안 함

구분	분산 없이 (Redis 없이)	분산 시 (Redis 사용)
서버 1대	1,000 req/min	1,000 req/min
서버 3대	3,000 req/min	1,000 req/min
확장성	서버 추가 시 한도 증가	서버 추가해도 한도 유지

기술 스택

분야	스택
Backend	Java 17, Spring Boot 3.x, Spring Cloud Gateway, Eureka, Config, Resilience4j
통신	Kafka (async 이벤트), OpenFeign (sync 호출), WebSocket (실시간 알림)
데이터	PostgreSQL + Liquibase, Redis, Kafka
Observability	Prometheus, Grafana, Loki, Promtail, Alertmanager, Zipkin
기타	Docker, Testcontainers, GitHub Actions

최고 — 팀원 모두의 피드백 반영

프로젝트를 마무리 하면서 팀원들로 부터 받았던 피드백을 정리하고 어떻게 반영했는지 표기 합니다. (2026.03.31)

김 팀원

- 기술 스택 정리 : 기술 스택 정리 부분에 Lombok, Spring Security, JWT 추가 (2026.03.31)
- 프로젝트 구조 : 프로젝트 구조 부분에 Controller, Service, Repository 등 각 폴더 역할 설명 추가 (2026.03.31)
- 트러블슈팅 : 트러블슈팅 부분에 N+1 문제 해결 과정 추가 (2026.03.31)

이 팀원

- 테스트 코드 : 테스트 코드 부분에 단위 테스트 결과 추가 (2026.03.31)
- 배포 : 배포 부분에 Docker Compose 구성 설명 추가 (2026.03.31)

박 팀원

- 보안 : 보안 부분에 JWT 토큰 만료 시간 설정 이유 추가 (2026.03.31)
- 성능 : 성능 부분에 Redis 캐싱 적용 전후 비교 추가 (2026.03.31)
- 문서 : 전체 문서 오타 및 맞춤법 수정 (2026.03.31)

최 팀원

- 아키텍처 : 아키텍처 다이어그램 수정 (2026.03.31)
- 회고 : 프로젝트 회고 부분 작성 (2026.03.31)

01 · OVERVIEW

AI가 일하는 방식을 설계하는 것 — 하네스 엔지니어링

개발을 AI에게 맡긴다는 것이 아니라, **AI가 이 코드베이스 규칙을 벗어나지 못하도록 통제·검독하는 체계**를 설계한다. AI의 결과물은 빠르고 그럴듯하지만 **맥락을 모른 채 자주 틀리기에**, `.antcomp` 기상 주식 거래 MSA · 도메인 서비스 7개에 **규칙 문서(SSOT) · 자동 검토/테스트 스킬 · 가드레일**을 둘러 **AI의 출력은 사람의 검증을 통과해야만 코드베이스에 반영되도록** 했습니다. 코드의 **최종 판단과 책임은 항상 사람**이 집니다.

harness engineering — AI를 통제·감독하는 체계를 엔지니어링

반복 타이핑만 AI로 자리하되, **무엇을·언제 참조할지, 무엇을 실행하면 안 되는지, 무엇을 반드시 검증할지**를 사람이 문서화합니다. AI가 단독으로 코드를 확정하는 일은 없습니다 — 모든 출력은 규칙·자동 검토·팀도 검증을 거쳐 **사람이 최종 확인**합니다. 도구(맡을CLI)는 빠르게 바뀌지만, 규칙 문서·검토 스킬·가드레일이라는 **관리 체계**는 프로젝트를 넘어 남습니다.

- SSOT 규칙 문서 3종
- 자동 검토·검증 A · B · C
- 합격 판정은 사람이

02 · DEV CYCLE

개발 사이클 단계별로 도구를 나눠 쓴다

분석·구현·리뷰는 요구하는 판단이 다릅니다. 단계마다 도구를 분리하되, **설계·결정과 핵심 분기는 사람이 직접 운전**하고 AI의 출력은 그 자리에서 검증합니다.

분석 · 설계

Claude Code

루트 `CLAUDE.md` 의 책사고날 4레이어·네이밍 규약을 기준으로 호출 경로와 설계 대안을 교차 검증. 프로젝트 안에 (application/domain/외부 인터 두고 어댑터를 `infrastructure` 에 두는 흐름을 그대로 따름.

구현 · 테스트

Claude Code · 서브에이전트

보일러플레이트·반복 작성해만 서브에이전트를 띄고, **도메인 로직과 예민 흐름은 직접 작성·통제**. 시키 보 로만든 경계 조건 검증 기준을 다시 설계해 **그대로 받지 않고 반드시 손본다**.

리뷰

/revref · CodeRabbit

커밋 전 직접 만든 `/revref` 스킬이 컨벤션 위반을 검검단규칙 위임 SOR+ 수정 시점), PR 단계는 **CodeRabbit**이 동일 원칙으로 한 번 더. **반복 지적은 규칙 문서로 흡수**.

03 · RULES · SSOT

규칙을 단일 출처(SSOT) 문서로 주입한다

AI가 일관되게 일하려면 "무엇이 규칙인가"가 **한 곳에** 있어야 합니다. 세 문서가 서로를 참조하며 **구조 · 규칙 · 절차**를 나눠 가지고, 다리에 붓(`.coderabbit.yaml`)과 **동일한 기준**을 공유합니다.

구조 · 컨벤션

CLAUDE.md

책사고날 4레이어, **패키지·네이밍 표**, 계이트웨이 인증 흐름(X-User-Id 신뢰 경계), Config Server 의무화, `common` 모듈 규약.

코딩 규칙

coding-conventions.md

트랜잭션 범위(JPA 캡슐화 CQS·레이어 검색·MSA 경계 `record` VO·정식성분 산하·로깅 보안·common 재사용, 위반은 **심각도**로 분류.

절차

automation-workflow.md

코드 작업 후 **자동 검토(A) · 빌드/테스트/API(B) · Slack 보고(C)** 워크플로우의 트리거와 흐름을 정의. "언제 무엇을 자동으로 할지"를 명문화.

심각도 체계 — 리뷰가 무엇을 막고 무엇을 권하는가

🔴 **Request Changes** 반드시 수정

- 아키텍처 경계 위반 — DDD(레이어)/트랜잭션/이벤트 경계
- `@Transactional` 안에서 외부 API 호출(커넥션 점유)
- JPA 엔티티 `@Setter` / `@Data` — 상태 변경은 의미 있는 메서드로
- 1 트랜잭션 1 Aggregate · 분산 트랜잭션(2PC) 금지 → Kafka 최종 일관성
- 동시성 제어 누락 · 민감정보 평문 로그

🟡 **Comment** 개선 권장

- 성능 · 코드 품질 · 리팩토링
- VO/DTO(Event)는 `record` - Strong Type VO(primitive obsession 회피)
- N+1 회피 — 연관관계 `LAZY` 기본, 필요 시 fetch join

코드베이스 특수 예외는 규칙으로 흡수한다

일반론이 이 코드베이스엔 안 맞는 경우, **왜 예외인지를 규칙 문서에 직접 박아** AI 리뷰가 같은 지적을 반복하지 않게 합니다. 예: "모든 `record` 의 compact constructor에 검증을 넣으라"는 일반 규칙에 대해 —

conventions §4 — 명문화된 예외

- 서비스 간 **내부 이벤트 메시지** `record` 는 발행 측에서 검증되었다고 가정하므로 compact constructor 검증의 예외로 둔다.
- → AI리뷰 봇이 "검증 누락"으로 잘못 지적하지 않고, **의도한 설계**임을 근거로 확인할 수 있다.

04 · WORKFLOWS

작업 단계에 맞춰 자동으로 도는 워크플로우 A · B · C

규칙은 "지켜야 한다"로 끝나지 않게, **작업이 끝나면 자동으로 검토·검증·보고**가 돌도록 슬래시 스킬과 스크립트로 묶었습니다.

워크플로우	트리거	수단
A · 코드 작업 후 자동 검토	새 파일/메서드 추가, 50줄+ 수정, `common/` 변경	`/revref` 스킬
B · 빌드·테스트·API 실행	컨트롤 동작에 영향 주는 변경 완료	`/apitest` 스킬
C · 완료 후 Slack 보고	A/B 또는 주요 작업 완료	`notify-slack.sh`

A. /revref — 컨벤션 준수 자동 검토

`git diff` 변경분(`--context`)마다 지정 파일을 대상으로, **자잔할 리뷰 에이전트를 병렬 실행**해 위반을 심각도와 근거(`conventions §N`)와 함께 보고합니다. 같은 일반론 지적이면 규칙 문서로 흡수합니다.

병렬 검토 자원 — 안 아키텍처(에이어/DDD)(상,도가 간 Spring(JPA·tx)부합성 ① 동시성(6) ④ 타입 안전성·공통 모듈(§4,9), `--context` 모드에서는 정확성(`code-review`)보다 (`security-review`) 집중하지 동합체해 🔴/🟡 + **파일:라인 + 근거 → 제안** 형식으로 종합합니다.

/revref 실행 결과 — 자잔화 병렬 검토 통합 리뷰

WORKFLOW A

CLICK TO ZOOM

```
● revref 검토 — notification-service 알림 상태별 카운트 API (5개 파일)

  치게 사용

  검토 5 🔴
  검토 2 🟡
  검토 3 통과 (치게 적당 사용 없음)
  제안 1

  🔴 src/.../NotificationStatusCountResponse.java:26
  🔴 src/.../client 내부 생성 후 정상 동작 (LazyInitializationException 가능) — 트랜잭션 밖에서 AlertStatus 값 추가 시 total에 새 종목이 누락 (규칙 §4 위반 — 도메인 로직 누락)
  🔴 src/.../controller/NotificationController.java:41
  🔴 @GetMapping("/status-counts") 에 @Transactional(readOnly = true) 누락 — 읽기 전용 트랜잭션 미적용으로 커넥션 점유 시간 증가 (규칙 §3 위반)

  🟡 src/.../service/NotificationStatusQueryService.java:58
  🟡 NotificationStatusQueryService.java:58 — null 타겟팅 상태로 들어올 수 있는 변경 시 처리 (타입 일관 오케이는 정상적 처리로 설정)
  제안 1 notification.forStatus(severity, source) 형식 메서드로 이동

  검토 3 ✅
  ✅ src/.../domain/AlertStatus.java:19
  ✅ src/.../config/RedisConfig.java:33 — NotificationStatusDto Slack payload 전부 포함
  ✅ src/.../client 및 서비스 생성자 주입, 별도 처리
```

변경분별 자원별 에이전트가 병렬 검토해 🔴/🟡 심각도 · 파일:라인 · 근거(conventions §N)과 함께 보고한다.

B. /apitest — 빌드·부터 게이트웨이 사니리오까지

1-2 · 빌드 · 정위	3-4 · 인프라 · 시나리오	5 · 보존
gradlew	**docker · *.http**	**tests/results**
`build` 는 `test` → `test` 실패 시 항목별(스트 원인을 오케이고 **여기서 중단**, 결과 저장 후 실패 보고.	게이트웨이 (`8888/actuator/health` · 플랫 후 `scenarios.http` 를 **게이트웨이 기준**으로 실행, 단계별 상태코드·본문으로 통과/실패를 판정.	타임스탬프 폴더에 `summary.md` 표요 장·원인 저장. **최근-시크릿은 마스킹**, 결과물은 `.gitignore` .

/apitest 실행 결과 — 빌드 → 테스트 → 게이트웨이 사니리오

WORKFLOW B

CLICK TO ZOOM

```
apitest 검토 — notification-service 상태별 카운트 API

1) 빌드 ./gradlew :notification-service:build -x test ✅ SUCCESSFUL
2) 단위 테스트 :apps:notification-service:test ✅ 34 SOURCED (통합 시에스 사이트 스스 없음)
3) 인프라 docker compose -f infra/docker-compose.yml up -d postgres redis kafka ✅ 3 healthy
4) 서비스 ./gradlew :apps:notification-service:bootRun & (로그 tail) — 8081/actuator/health 🟢 UP
5) API 실행 — 8/8 ✅

| # | 케이스 | 엔드포인트 | 기대 | 실제 | 결과 |
|---|---|---|---|---|---|
| 01 | 정상 집계 | …/admin/stats | 8/3/5/3/5 | 8/3/5/3/5 | ✅ |
| 02 | severity=CRITICAL | …/severity=CRITICAL | 3/3/3/3/0 | 3/3/3/3/0 | ✅ |
| 03 | source=PROMETHEUS | …/source=PROMETHEUS | 5/5/3/4/0 | 5/5/3/4/0 | ✅ |
| 04 | 두 필터 동시 | …/severity=CRITICAL&source=PROMETHEUS | 2/3/3/0/0 | 2/3/3/0/0 | ✅ |
| 05 | 경계 0건 | …/severity=INFO&source=GRAFANA | 0/0/0/0/0 | 0/0/0/0/0 | ✅ |
| 06 | 잘못된 enum | …/severity=DOGGO | 400 INVALID_INPUT | 400 INVALID_INPUT | ✅ |
| 07 | 목록 일관성(안 맞) | /admin?page=0 | totalElements=8 | 8 | ✅ |
| 08 | 목록 일관성(CRITICAL) | /admin?severity=CRITICAL | totalElements=3 | 3 | ✅ |
```

빌드·단위 테스트 후 게이트웨이 띄고 플랫 → 사니리오(http)를 단계별 상태코드·본문으로 **통과/실패** 판정합니다.

C. notify-slack.sh — 작업을 채지 않는 보고

자부 전송이므로 처음에는 보고 대상 내용을 사용자에게 확인받고, 사소한 변경분(사닝이)은 보고하지 않습니다.

notify-slack.sh 보고 — 작업 완료 Slack 알림

WORKFLOW C

```
☆ #  dev-report

  메시지   앱에스 추가   +

  dev-report 앱  오전 1:00                                  오늘
  ✅ apitest: notification-service 상태별 카운트 API

  # apitest: notification-service — 알림 상태별 카운트 API

  - 대상: `GET /api/notifications/admin/stats` (신규) + 회귀: `GET /api/notifications/admin`
  - 일시: 2026-06-29 00:56 (KST)
  - 변경 파일: NotificationController, NotificationStatsRequest(신규), NotificationStatsQuery(신규), NotificationStatusCountResponse(신규), NotificationQueryService, NotificationRepository(port), NotificationRepositoryImpl, NotificationQueryRepository(GROUP BY 집계)
```

성공/실패 · 제목 · 요약을 Incoming Webhook으로 전송한다. 웹훅 미설정 혹은 실패 전송 오류 시 경고만 출력하고 **작업을 깨지 않는다**.

05 · GUARDRAILS · CONTEXT

가드레일과 컨텍스트를 설계한다

생성을 빠르게 하되, **위험한 자동화는 막고(가드레일)** AI가 **틀린 맥락으로 추측하지 않게(컨텍스트)** 합니다.

가드레일 — 무엇을 못 하게 하나

- **합격 판정은 사람이** — 검토·테스트는 자동화하되 통과 여부는 직접 결정. 🔴 항목은 반드시 수정 후 **빌드로 재검증**
- **외부 전송은 확인 후** — Slack 보고는 처음에 사용자 승인
- **시크릿·커밋 스캔** — 테스트 결과로 그에 JWT/ `sk-` / `xox-` 키 정규식 패턴이 보이면 경고(§8)
- 테스트 산출물(`tests/results/`)은 **커밋 대상 아님** (.gitignore)
- 신뢰 경계 — 도메인은 서비스는 JWT를 직접 검증하지 않고 게이트웨이가 주입한 `X-User-Id` 만 신뢰

컨텍스트 — 무엇을 언제 보게 하나

- **SSOT 상호참조** — 세 문서가 서로를 가리켜 중복과 모순을 방지
- **설정 파부터 인지** — "이 레포에서 `application.yaml` 찾지 말 것", 실제 설정은 Config 레포(`dev`)
- **작업 유형별 참고서 라우팅** — RAG 적용 → `rag-architecture.md` , 장애 작성 → `alops-architecture.md`
- 위험성 큰 정보(스키마·설정)는 문서에 박제하지 않고 그때그때 실측

06 · PRINCIPLE

왜 이렇게까지 하는가

AI 생성물은 빠르지만 **이 코드베이스의 맥락을 모른 채 그럴듯하지만 틀린 코드**를 자신 있게 내놓습니다. 그 실수가 코드베이스에 들어오지 못하도록 규칙(SSOT)·자동 검토/검증·가드레일을 **여러 겹의 관문**을 두고, 사람은 생성이 아니라 **감독과 최종 판단**에 시간을 씁니다. AI는 빨라지되 **사람의 통제 밖에서 코드를 확정하지 못합니다.** 이 관리 체계는 모델·도구가 바뀌어도 남습니다.

AI에게 개발을 일임하지 않는다 — 통제하고 감독한다.